CODE 1:

```
-- CREATE TABLE
CREATE   TABLE   EMPLOYEE   (
 empId INTEGER PRIMARY KEY,
 first_name TEXT NOT NULL,
 last_name TEXT NOT NULL,
 dept TEXT NOT NULL,
 hire_date TEXT NOT NULL,
 salary INTEGER NOT NULL
);

-- INSERT DATA
INSERT INTO EMPLOYEE VALUES (101, 'Clark', 'John','Sales','2024-02-23','20000');
INSERT INTO EMPLOYEE VALUES (102, 'Raj', 'Kumar','Marketing','2023-04-23','30000');
INSERT INTO EMPLOYEE VALUES (103, 'Michael', 'Raj','IT','2022-06-23','50000');

-- DISPLAY ALL
SELECT * FROM EMPLOYEE;

-- SELECT SPECIFIC COLUMNS
SELECT first_name, last_name, dept FROM EMPLOYEE;

-- FILTER DATA
SELECT * FROM EMPLOYEE WHERE dept = 'Marketing';

-- UPDATE
UPDATE EMPLOYEE SET last_name = 'Raj' WHERE empId = 101;
SELECT * FROM EMPLOYEE;

-- DELETE ONE
DELETE FROM EMPLOYEE WHERE empId = 102;
```

```
SELECT * FROM EMPLOYEE;

-- DELETE BY CONDITION
DELETE FROM EMPLOYEE WHERE hire_date = '2024-02-23';
SELECT * FROM EMPLOYEE;

-- ALTER TABLE
ALTER TABLE EMPLOYEE ADD DOB INTEGER;

-- VIEW STRUCTURE (SQLite)
PRAGMA table_info(EMPLOYEE);

-- TRUNCATE (SQLite alternative)
DELETE FROM EMPLOYEE;
SELECT * FROM EMPLOYEE;

-- DROP TABLE
DROP TABLE EMPLOYEE;
```

OUTPUT:

empId	first_name	last_name	dept	hire_date	salary
101	Clark	John	Sales	2024-02-23	20000
102	Raj	Kumar	Marketing	2023-04-23	30000
103	Michael	Raj	IT	2022-06-23	50000

first_name	last_name	dept
Clark	John	Sales
Raj	Kumar	Marketing
Michael	Raj	IT

empId	first_name	last_name	dept	hire_date	salary
102	Raj	Kumar	Marketing	2023-04-23	30000

empId	first_name	last_name	dept	hire_date	salary
101	Clark	Raj	Sales	2024-02-23	20000
102	Raj	Kumar	Marketing	2023-04-23	30000
103	Michael	Raj	IT	2022-06-23	50000

empId	first_name	last_name	dept	hire_date	salary
101	Clark	Raj	Sales	2024-02-23	20000
103	Michael	Raj	IT	2022-06-23	50000

empId	first_name	last_name	dept	hire_date	salary
103	Michael	Raj	IT	2022-06-23	50000

cid	name	type	notnull	dflt_value	pk
0	empId	INTEGER	0		1
1	first_name	TEXT	1		0
2	last_name	TEXT	1		0
3	dept	TEXT	1		0
4	hire_date	TEXT	1		0
5	salary	INTEGER	1		0
6	DOB	INTEGER	0		0

CODE 2:

```
-- CREATE TABLE
CREATE TABLE EMPLOYEE (
 empId INTEGER PRIMARY KEY,
 first_name TEXT NOT NULL,
 last_name TEXT NOT NULL,
 dept TEXT NOT NULL,
 hire_date TEXT NOT NULL,
 salary INTEGER NOT NULL
);
-- INSERT RECORDS
INSERT INTO EMPLOYEE VALUES (101, 'Clark', 'John','Sales','2024-02-23','20000');
INSERT INTO EMPLOYEE VALUES (102, 'Raj', 'Kumar','Marketing','2023-04-23','30000');
INSERT INTO EMPLOYEE VALUES (103, 'Michael', 'Raj','IT','2022-06-23','50000');

--    READ    (DISPLAY    ALL)
SELECT * FROM EMPLOYEE;

-- READ (SPECIFIC COLUMNS)
SELECT first_name, last_name, dept FROM EMPLOYEE;

-- READ (FILTER)
SELECT * FROM EMPLOYEE WHERE dept = 'Marketing';

-- UPDATE
UPDATE EMPLOYEE SET last_name = 'Raj' WHERE empId = 101;
SELECT * FROM EMPLOYEE;

-- DELETE ONE RECORD
DELETE FROM EMPLOYEE WHERE empId = 102;
SELECT * FROM EMPLOYEE;
```

```
-- DELETE USING CONDITION
DELETE FROM EMPLOYEE WHERE hire_date = '2024-02-23';
SELECT * FROM EMPLOYEE;

-- ALTER TABLE
ALTER TABLE EMPLOYEE ADD DOB INTEGER;

-- VIEW STRUCTURE (SQLite)
PRAGMA table_info(EMPLOYEE);

-- DELETE ALL (TRUNCATE ALTERNATIVE)
DELETE FROM EMPLOYEE;
SELECT * FROM EMPLOYEE;

-- DROP TABLE
DROP TABLE EMPLOYEE;
```

OUTPUT:

empId	first_name	last_name	dept	hire_date	salary
101	Clark	John	Sales	2024-02-23	20000
102	Raj	Kumar	Marketing	2023-04-23	30000
103	Michael	Raj	IT	2022-06-23	50000

first_name	last_name	dept
Clark	John	Sales
Raj	Kumar	Marketing
Michael	Raj	IT

empId	first_name	last_name	dept	hire_date	salary
102	Raj	Kumar	Marketing	2023-04-23	30000

empId	first_name	last_name	dept	hire_date	salary
101	Clark	Raj	Sales	2024-02-23	20000

empId	first_name	last_name	dept	hire_date	salary
102	Raj	Kumar	Marketing	2023-04-23	30000
103	Michael	Raj	IT	2022-06-23	50000

empId	first_name	last_name	dept	hire_date	salary
101	Clark	Raj	Sales	2024-02-23	20000
103	Michael	Raj	IT	2022-06-23	50000

empId	first_name	last_name	dept	hire_date	salary
103	Michael	Raj	IT	2022-06-23	50000

cid	name	type	notnull	dflt_value	pk
0	empId	INTEGER	0		1
1	first_name	TEXT	1		0
2	last_name	TEXT	1		0
3	dept	TEXT	1		0
4	hire_date	TEXT	1		0
5	salary	INTEGER	1		0
6	DOB	INTEGER	0		0

CODE 3:

```
-- CREATE TABLES
CREATE TABLE Author (
 AuthorID INT PRIMARY KEY,
 AuthorName VARCHAR(255),
 BirthDate DATE
);

CREATE TABLE Book (
 ISBN VARCHAR(20) PRIMARY KEY,
 Title VARCHAR(255),
 AuthorID INT,
 Genre VARCHAR(50),
 FOREIGN KEY (AuthorID) REFERENCES Author(AuthorID)
);

CREATE TABLE Member (
 MemberID INT PRIMARY KEY,
 MemberName VARCHAR(255),
 Email VARCHAR(255)
);

CREATE TABLE Transactions (
 TransactionID INT PRIMARY KEY,
 BookID VARCHAR(20),
 MemberID INT,
 CheckoutDate DATE,
 ReturnDate DATE,
 FOREIGN KEY (BookID) REFERENCES Book(ISBN),
 FOREIGN KEY (MemberID) REFERENCES Member(MemberID)
);
```

```sql
-- INSERT DATA

INSERT INTO Author VALUES (1, 'John Doe', '1990-01-01');

INSERT INTO Book VALUES ('1234567890', 'Sample Book', 1, 'Fiction');

INSERT INTO Member VALUES (101, 'Alice Smith', 'alice@example.com');
INSERT INTO Member VALUES (102, 'Reema Teraja', 'reema@example.com');
INSERT INTO Member VALUES (103, 'David Raj', 'david@example.com');

INSERT INTO Transactions VALUES (1, '1234567890', 101, '2024-02-28', '2024-03-15');
INSERT INTO Transactions VALUES (2, '1234567890', 102, '2024-02-20', '2024-03-08');
INSERT INTO Transactions VALUES (3, '1234567890', 103, '2024-01-28', '2024-02-15');

-- OUTPUT
SELECT * FROM Author;
SELECT * FROM Book;
SELECT * FROM Member;
SELECT * FROM Transactions;

-- JOIN OUTPUT (IMPORTANT)
SELECT
 m.MemberName,
 b.Title,
 t.CheckoutDate,
 t.ReturnDate
FROM Transactions t
JOIN Member m ON t.MemberID = m.MemberID
JOIN Book b ON t.BookID = b.ISBN;
```

OUTPUT:

AuthorID	AuthorName	BirthDate
1	John Doe	1990-01-01

ISBN	Title	AuthorID	Genre
1234567890	Sample Book	1	Fiction

MemberID	MemberName	Email
101	Alice Smith	alice@example.com
102	Reema Teraja	reema@example.com
103	David Raj	david@example.com

TransactionID	BookID	MemberID	CheckoutDate	ReturnDate
1	1234567890	101	2024-02-28	2024-03-15
2	1234567890	102	2024-02-20	2024-03-08
3	1234567890	103	2024-01-28	2024-02-15

MemberName	Title	CheckoutDate	ReturnDate
Alice Smith	Sample Book	2024-02-28	2024-03-15
Reema Teraja	Sample Book	2024-02-20	2024-03-08
David Raj	Sample Book	2024-01-28	2024-02-15

CODE 4:

```
-- CREATE TABLES
CREATE TABLE events (
 event_id INTEGER PRIMARY KEY,
 eventName TEXT,
 date TEXT,
 organizer TEXT,
 venue TEXT
);

CREATE TABLE organizers (
 organizer_id INTEGER PRIMARY KEY,
 organizerName TEXT,
 contactPerson TEXT
);

CREATE TABLE attendees (
 attendee_id INTEGER PRIMARY KEY,
 attendeeName TEXT,
 event TEXT
);

CREATE TABLE venues (
 venue_id INTEGER PRIMARY KEY,
 venueName TEXT,
 capacity INTEGER
);

-- INSERT DATA
INSERT INTO events VALUES (1, 'Tech Conference', '2024-04-15', 'Tech Events LLC', 'Conference Center A');
```

```
INSERT INTO organizers VALUES (1, 'Music Festivals Inc', 'Emily Adams');

INSERT INTO attendees VALUES (1, 'Charlie', 'Tech Conference');

INSERT INTO venues VALUES (1, 'Exhibition Hall', 500);

-- OUTPUT
SELECT * FROM events;
SELECT * FROM organizers;
SELECT * FROM attendees;
SELECT * FROM venues;

-- UPDATE
UPDATE events SET venue = 'Conference Center B' WHERE event_id = 1;
SELECT * FROM events;

-- DELETE
DELETE FROM attendees WHERE attendeeName = 'Bob';
SELECT * FROM attendees;
```

OUTPUT:

event_id	eventName	date	organizer	venue
1	Tech Conference	2024-04-15	Tech Events LLC	Conference Center A

organizer_id	organizerName	contactPerson
1	Music Festivals Inc	Emily Adams

attendee_id	attendeeName	event
1	Charlie	Tech Conference

venue_id	venueName	capacity
1	Exhibition Hall	500

event_id	eventName	date	organizer	venue
1	Tech Conference	2024-04-15	Tech Events LLC	Conference Center B

attendee_id	attendeeName	event
1	Charlie	Tech Conference

CODE 5:

```
-- DELETE OLD TABLES (IMPORTANT)
DROP TABLE IF EXISTS products;
DROP TABLE IF EXISTS orders;

-- CREATE TABLES
CREATE TABLE products (
 product_id INTEGER PRIMARY KEY,
 name TEXT,
 price REAL
);

CREATE TABLE orders (
 order_id INTEGER PRIMARY KEY,
 product_id INTEGER,
 quantity INTEGER,
 total_price REAL
);

-- INSERT DATA
INSERT INTO products VALUES (1, 'Product A', 19.99);
INSERT INTO products VALUES (2, 'Product B', 29.99);

-- VIEW PRODUCTS
SELECT * FROM products;

-- INSERT ORDERS
INSERT INTO orders VALUES (101, 1, 2, 39.98);
INSERT INTO orders VALUES (102, 2, 1, 29.99);

-- VIEW ORDERS
```

SELECT * FROM orders;

OUTPUT:

product_id	name	price
1	Product A	19.99
2	Product B	29.99

order_id	product_id	quantity	total_price
101	1	2	39.98
102	2	1	29.99

CODE 6:

```
-- CREATE TABLE (Simulating Cache + Database)
CREATE TABLE blog_posts (
 post_id INTEGER PRIMARY KEY,
 title TEXT,
 content TEXT
);

CREATE TABLE cache (
 post_id INTEGER PRIMARY KEY,
 title TEXT,
 content TEXT
);

-- INSERT DATA INTO MAIN DATABASE
INSERT INTO blog_posts VALUES (1, 'Title of Post 1', 'Content of Post 1');
INSERT INTO blog_posts VALUES (2, 'Title of Post 2', 'Content of Post 2');

-- STEP 1: FETCH (CACHE MISS → FROM DB)
SELECT * FROM blog_posts WHERE post_id = 1;

-- STORE INTO CACHE
INSERT INTO cache
SELECT * FROM blog_posts WHERE post_id = 1;

-- STEP 2: FETCH AGAIN (CACHE HIT)
SELECT * FROM cache WHERE post_id = 1;

-- ANOTHER POST (CACHE MISS)
SELECT * FROM blog_posts WHERE post_id = 2;
```

```
-- STORE IN CACHE
INSERT INTO cache
SELECT * FROM blog_posts WHERE post_id = 2;

-- FETCH FROM CACHE (CACHE HIT)
SELECT * FROM cache WHERE post_id = 2;
```

OUTPUT:

post_id	title	content
1	Title of Post 1	Content of Post 1
post_id	title	content
1	Title of Post 1	Content of Post 1
post_id	title	content
2	Title of Post 2	Content of Post 2
post_id	title	content
2	Title of Post 2	Content of Post 2

CODE 7:

```
-- DROP OLD TABLES (to avoid errors)
DROP TABLE IF EXISTS users;
DROP TABLE IF EXISTS roles;

-- CREATE ROLES TABLE (RBAC)
CREATE TABLE roles (
 role_id INTEGER PRIMARY KEY,
 role_name TEXT
);

-- CREATE USERS TABLE
CREATE TABLE users (
 user_id INTEGER PRIMARY KEY,
 username TEXT UNIQUE,
 password TEXT,
 role_id INTEGER,
 account_number TEXT,
 FOREIGN KEY (role_id) REFERENCES roles(role_id)
);

-- INSERT ROLES
INSERT INTO roles VALUES (1, 'Admin');
INSERT INTO roles VALUES (2, 'Customer');

-- INSERT USERS (PASSWORD STORED AS HASH SIMULATION)
INSERT INTO users VALUES (101, 'alice', 'hashed_password_123', 1, '1234567890123456');
INSERT INTO users VALUES (102, 'bob', 'hashed_password_456', 2, '9876543210987654');

-- VIEW USERS
SELECT * FROM users;
```

```
-- LOGIN SIMULATION (AUTHENTICATION)
SELECT * FROM users
WHERE username = 'alice' AND password = 'hashed_password_123';

-- ACCESS CONTROL (CHECK ROLE)
SELECT u.username, r.role_name
FROM users u
JOIN roles r ON u.role_id = r.role_id;

-- UPDATE PASSWORD (POLICY SIMULATION)
UPDATE users
SET password = 'new_hashed_password'
WHERE username = 'bob';

SELECT * FROM users;

-- DELETE USER (SECURITY CONTROL)
DELETE FROM users WHERE username = 'bob';

SELECT * FROM users;
```

OUTPUT:

user_id	username	password	role_id	account_number
101	alice	hashed_password_123	1	1234567890123456
102	bob	hashed_password_456	2	9876543210987654

user_id	username	password	role_id	account_number
101	alice	hashed_password_123	1	1234567890123456

username	role_name
alice	Admin
bob	Customer

user_id	username	password	role_id	account_number
101	alice	hashed_password_123	1	1234567890123456
102	bob	new_hashed_password	2	9876543210987654
user_id	username	password	role_id	account_number
101	alice	hashed_password_123	1	1234567890123456